

Arquitectura de Computadoras
(CS3051)

Diseño e implementación de un procesador RISC-V superescalar Dual-Issue con optimización microarquitectónica

Proyecto 2 - Segunda entrega

Alvarado León, Adriana Celeste
Daza Yalta, Gino Jesús
Ccolcca Flores, Marylin Regina

Profesor: Williams Valencia , Carlos Eduardo
10 de noviembre del 2026

Índice

1. Introducción **3**

2. Arquitectura Dual-Issue **3**
2.1. Organización general 3
2.2. Datapath completo 3
2.3. Controller 8
2.4. Módulos modificados 8

3. Issue Unit **8**
3.1. Condiciones de emisión 8
3.2. Asignación de recursos 9
3.3. Emisión parcial y reagrupamiento 10

4. Forwarding, hazards y control de branches **10**
4.1. Forwarding entre vías y etapas 10
4.2. Stalls por load-use 10
4.3. Control de branches y flushing 11

5. Programas de prueba y waveforms **11**
5.1. Programa 1: dos operaciones ALU independientes 12
5.2. Programa 2: RAW dentro del par y forwarding entre vías 12
5.3. Programa 3: load-use y conflictos del puerto de memoria 14
5.4. Programa 4: branches en ambas posiciones y flushing 16
5.5. Programa 5: dos escrituras en el Register File 18
5.6. Programa 6: reagrupamiento y orden del programa 20
5.7. Programa 7: conflicto en la Branch Unit y jal en el slot 1 21
5.8. Resumen de señales 24

6. Comparación preliminar frente al Single-Issue **25**
6.1. Metodología 25
6.2. Resultados 25
6.3. Análisis 25

7. Conclusiones **26**

1. Introducción

En la primera entrega se implementó el pipeline RV32I de cinco etapas con forwarding, stalling y flushing, y se inició la extensión superescalar con Dual Fetch y Dual Decode; en ese momento solo la vía 0 ejecutaba instrucciones. En esta segunda entrega se completa el procesador **Dual-Issue In-Order**: las dos vías llegan hasta Writeback, cada una con su propia ALU, y comparten un único puerto de memoria de datos y una única unidad de resolución de saltos (*Branch Unit*).

Para decidir cuántas instrucciones del par avanzan en cada ciclo se agregó una **Issue Unit**, y la Hazard Unit se amplió con forwarding entre vías, stalls por load-use en cualquiera de las dos vías, reagrupamiento de la instrucción que no pudo emitirse y anulación selectiva cuando se toma un salto. Todo el diseño está en `designs/dualissue/`.

El informe describe la arquitectura (Sección 2), la Issue Unit (Sección 3), el manejo de hazards (Sección 4), las waveforms de los siete programas de prueba (Sección 5) y una comparación preliminar de ciclos e IPC frente al pipeline Single-Issue (Sección 6).

2. Arquitectura Dual-Issue

2.1. Organización general

El procesador conserva las cinco etapas IF, ID, EX, MEM y WB. En cada ciclo se buscan y decodifican dos instrucciones consecutivas: la del **slot 0** (dirección `PCD_0`, la más antigua) y la del **slot 1** (`PCD_0`+4, la más joven). La instrucción del slot 0 siempre viaja por la vía 0 (*lane 0*) y la del slot 1 por la vía 1 (*lane 1*), por lo que el orden del programa se mantiene: en cualquier etapa, la vía 0 contiene la instrucción más antigua del par.

La Figura 1 muestra cómo se conectan los módulos. `top.v` contiene el procesador (`riscv.v`) y las memorias externas: la memoria de instrucciones con dos puertos de lectura (`RD1`, `RD2`) y la memoria de datos de un solo puerto. Dentro de `riscv.v` se instancian cuatro bloques:

- **Controller** (`controller.v`): un decodificador (`maindec` + `aludec`) por vía y los registros de control de cada vía.
- **Datapath** (`datapath.v`): las dos vías, el Register File de 4 lecturas y 2 escrituras, la Branch Unit compartida y el multiplexor del puerto de memoria.
- **Issue Unit** (`issue.v`, nuevo): examina el par en Decode y decide si se emiten las dos instrucciones, solo la del slot 0 o ninguna.
- **Hazard Unit** (`hazard.v`): forwarding de ambas vías, stalls, reagrupamiento y flushes.

2.2. Datapath completo

La Figura 2 presenta el datapath completo. A continuación se describen los cambios respecto a la versión preliminar de E1, etapa por etapa.

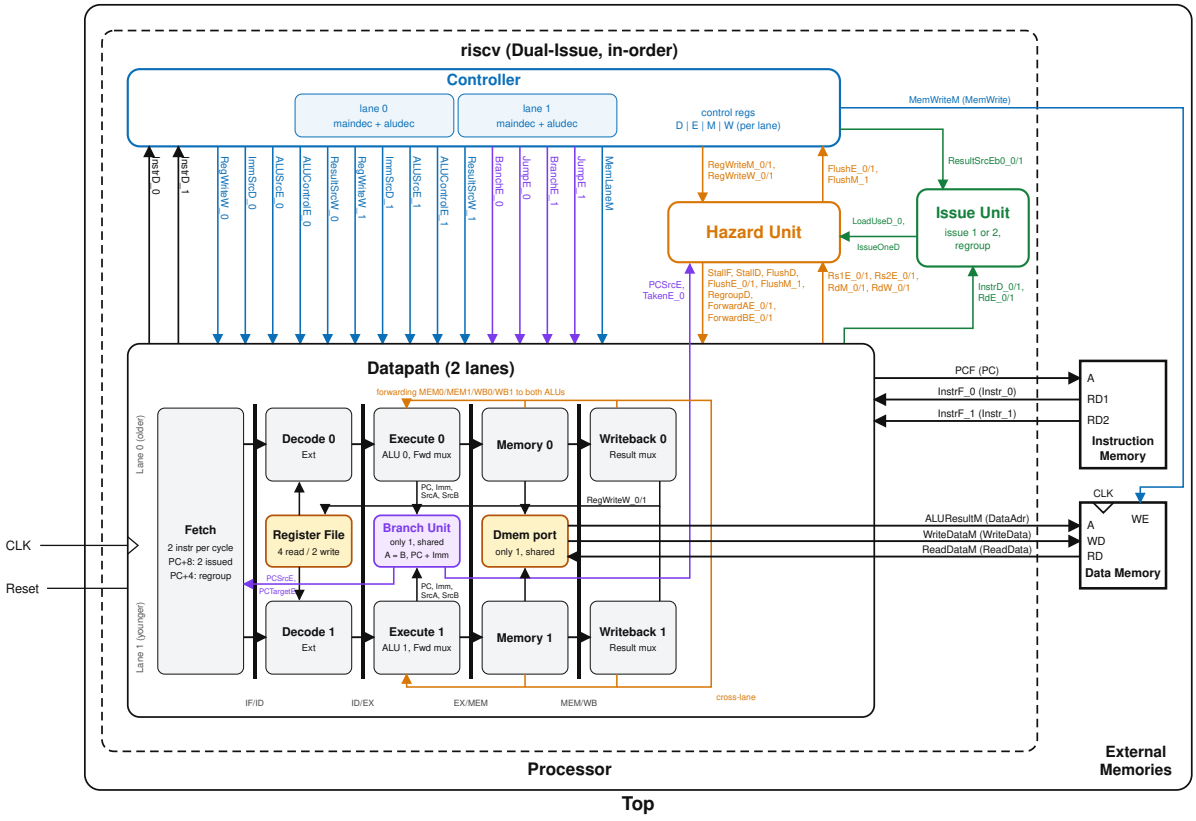


Figura 1: Integración de los módulos del procesador Dual-Issue (`top.v` y `riscv.v`).

Fetch. La memoria de instrucciones entrega `InstrF_0` = `RAM[PC]` e `InstrF_1` = `RAM[PC+4]`. El multiplexor del PC (`pcmux`) ahora usa sus tres entradas:

Condición	PC siguiente	Caso
`PCSrcE`= 1	`PCTargetE`	salto tomado en EX (prioridad)
`RegroupD`= 1	`PCF`+4	solo se emitió el slot 0
otro caso	`PCF`+8	se emitieron ambas instrucciones

El selector es `{~PCSrcE & ~RegroupD, PCSrcE}`. Además, `StallF` congela el PC durante un stall por load-use.

Registro IF/ID y reagrupamiento. El registro `regD` (192 bits) guarda las dos instrucciones, sus PC y sus PC+4. Delante de él se agregaron multiplexores controlados por `RegroupD`: cuando solo se emite el slot 0, la instrucción que esperaba en el slot 1 pasa a ser el nuevo slot 0 y la primera instrucción buscada (`InstrF_0`) pasa a ser el nuevo slot 1:

```
wire [31:0] InstrN_0 = RegroupD ? InstrD_1 : InstrF_0;
wire [31:0] InstrN_1 = RegroupD ? InstrF_0 : InstrF_1;
wire [31:0] PCN_0    = RegroupD ? PCD_1    : PCF;
wire [31:0] PCN_1    = RegroupD ? PCF      : PCPlus4F;
```

Como en ese ciclo el PC avanza solo 4, `PCF` es exactamente la dirección siguiente a `PCD_1`, y el nuevo par sigue siendo de dos instrucciones consecutivas. Así ninguna instrucción se pierde ni se ejecuta dos veces.

Decode. Cada vía tiene su decodificador de control, su generador de inmediato (`ext_0`, `ext_1`) y dos puertos de lectura del Register File (`rd1`/`rd2` para la vía 0 y `rd4`/`rd5` para la vía 1). Los registros ID/EX están separados por vía (`regE_0`, `regE_1`) y tienen limpieza independiente (`FlushE_0`, `FlushE_1`), lo que permite convertir en burbuja solo la vía 1 cuando el slot 1 no se emite.

Execute. Cada vía tiene su ALU (`alu_0`, `alu_1`) y dos multiplexores de forwarding de cinco entradas (`mux5`): Register File, `ResultW_0`, `ResultW_1`, `ALUResultM_0` y `ALUResultM_1`. De esta forma cualquier operando puede tomar el resultado de cualquiera de las dos vías en MEM o WB.

La resolución de saltos ya no está duplicada: se agregó el módulo `branchunit.v`, el único recurso que compara operandos (`beq`) y calcula la dirección destino. Como la Issue Unit nunca emite dos instrucciones de control en el mismo par, a lo sumo una vía tiene un salto en EX. La señal `BrLaneE` = `BranchE_1` $\vee$ `JumpE_1` indica cuál, y con ella cuatro multiplexores eligen el PC, el inmediato y los dos operandos (ya adelantados) de esa vía. La unidad tiene un solo comparador y un solo sumador:

$$\texttt{PCTargetE} = \texttt{PCE} + \texttt{ImmExtE} \tag{1}$$

$$\texttt{PCSrcE} = (\texttt{BranchE} \wedge (\texttt{SrcAE} = \texttt{SrcBE})) \vee \texttt{JumpE} \tag{2}$$

$$\texttt{TakenE_0} = \texttt{PCSrcE} \wedge \neg\texttt{BrLaneE} \tag{3}$$

`TakenE_0` indica que el salto tomado está en la vía 0; en ese caso la instrucción de la vía 1 es más joven y debe anularse (Sección 4).

Memory. Hay un único puerto de memoria de datos. El Controller genera `MemLaneM`, que vale 1 cuando la instrucción de memoria está en la vía 1 (`MemWriteM_1` o `ResultSrcM_1=01`); con ella los multiplexores `memaddrmux` y `memdatamux` entregan a la memoria la dirección y el dato de esa vía. La escritura se habilita con `MemWriteM` = `MemWriteM_0` $\vee$ `MemWriteM_1`. Esto funciona porque la Issue Unit garantiza que nunca hay dos instrucciones de memoria en el mismo par. El dato leído (`ReadDataM`) llega a los dos registros MEM/WB y solo lo usa la vía que tiene el `lw`. El registro EX/MEM de la vía 1 (`regM_1`) se puede limpiar con `FlushM_1`.

Writeback. Cada vía tiene su multiplexor de resultado (`resultmux_0`, `resultmux_1`) y su puerto de escritura en el Register File (`we3/a3/wd3` y `we6/a6/wd6`). Ambos escriben en el flanco de bajada; si las dos vías escriben el mismo registro, el puerto de la vía 1 se aplica después y gana, que es lo correcto porque esa instrucción es la más joven.

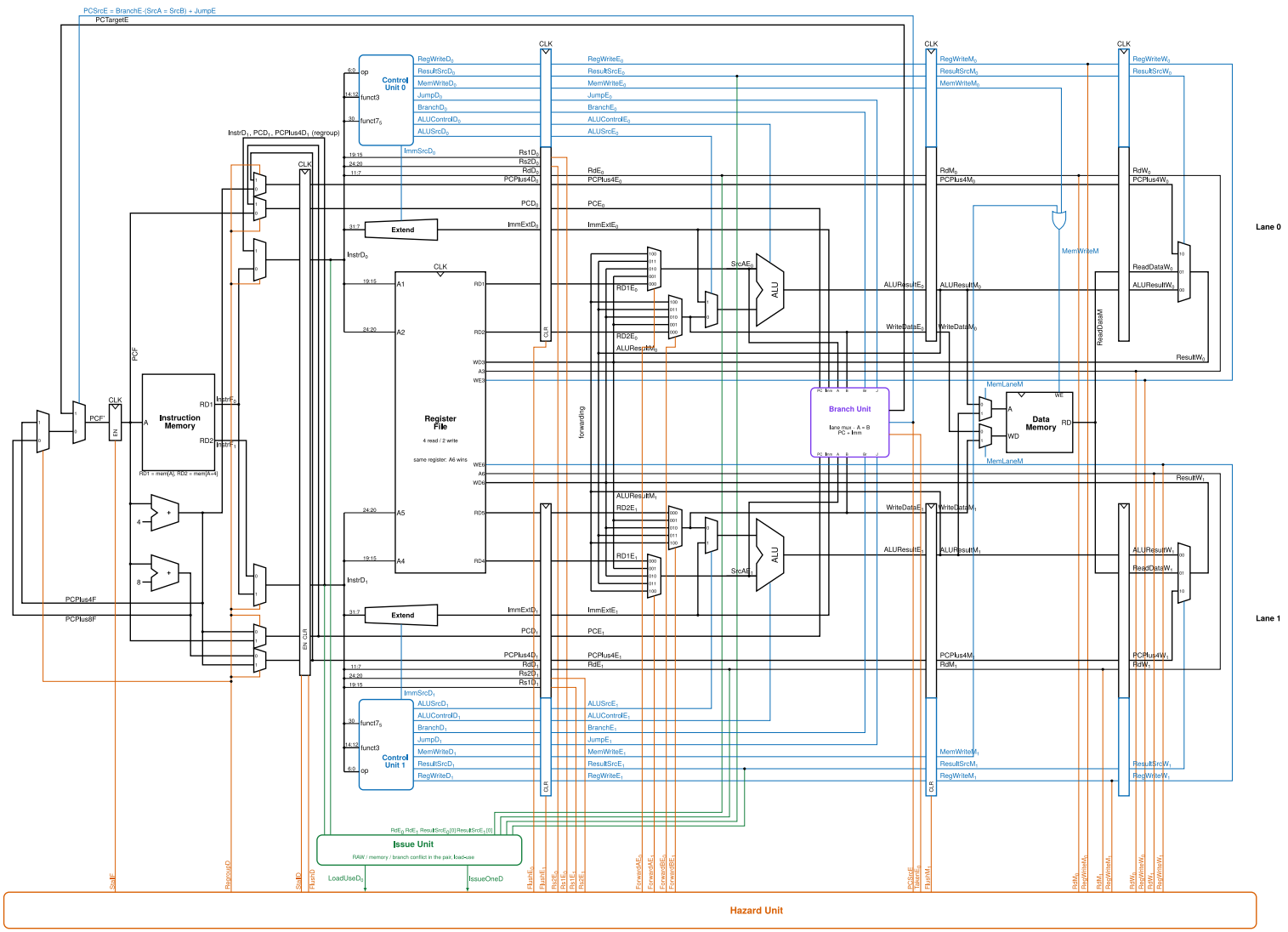


Figura 2: Datapath completo del procesador Dual-Issue. Arriba la vía 0, abajo la vía 1; al centro el Register File (4 lecturas, 2 escrituras), la Branch Unit y el puerto de memoria compartidos. En verde la Issue Unit y en naranja la Hazard Unit.

2.3. Controller

El Controller mantiene los dos decodificadores de E1 (`md_0`/`ad_0` y `md_1`/`ad_1`), y ahora los registros de control de la vía 1 llegan hasta WB. Los cambios respecto a E1 son:

- **Saltos fuera del Controller.** Antes calculaba `PCSrcE`; ahora solo exporta `BranchE_0/1` y `JumpE_0/1` a la Branch Unit, que es la que decide.
- **Limpieza por vía.** `regE_0` se limpia con `FlushE_0`, `regE_1` con `FlushE_1` y `regM_1` con `FlushM_1`. Al limpiarse, `RegWrite` y `MemWrite` quedan en 0, así que una instrucción anulada no escribe registros ni memoria.
- **Puerto de memoria.** Genera `MemWriteM` (OR de ambas vías) y `MemLaneM`.
- **Señales para hazards.** Envía `ResultSrcEb0_0/1` (hay un `lw` en EX) a la Issue Unit y `RegWriteM_0/1`, `RegWriteW_0/1` a la Hazard Unit.

2.4. Módulos modificados

Módulo	Cambio en E2
`issue.v`	**Nuevo.** Decide la emisión del par (`IssueOneD`) y detecta load-use en el slot 0 (`LoadUseD_0`).
`branchunit.v`	**Nuevo.** Branch Unit compartida: un comparador y un sumador, operandos de la vía que tiene el salto.
`mux5.v`	**Nuevo.** Multiplexor de 5 entradas para el forwarding.
`datapath.v`	Vía 1 completa (ALU, forwarding, EX/MEM, MEM/WB, WB); `pcmux` con PC+8; multiplexores de reagrupamiento en IF/ID; multiplexores del puerto de memoria; `regM_1` con clear.
`controller.v`	Registros de control de la vía 1 hasta WB, `MemLaneM`, `MemWriteM` compartido; la decisión de salto pasa a la Branch Unit.
`hazard.v`	Forwarding de 3 bits con prioridad entre vías; `RegroupD`, `FlushE_0/1`, `FlushM_1`.
`riscv.v`	Conecta la Issue Unit y las nuevas señales entre Controller, Datapath y Hazard Unit.
`imem.v`	Inicializa la memoria en cero, para que el segundo puerto no lea `x` al final del programa (se decodifica como burbuja).

Cuadro 1: Módulos nuevos y modificados respecto a la primera entrega.

`regfile.v` (4R/2W) y el IF/ID de 192 bits ya existían desde E1. `alu`, `aludec`, `maindec`, `extend`, `dmem`, `adder`, `mux2`, `mux3` y los registros `flop*` no cambiaron.

3. Issue Unit

3.1. Condiciones de emisión

La Issue Unit (`issue.v`) es combinacional y trabaja sobre el par que está en Decode. Primero identifica el tipo de cada instrucción a partir de su *opcode* y qué registros fuente

lee realmente (por ejemplo, un `addi` no lee `rs2` y un `jal` no lee ninguno), para no detectar dependencias falsas. Luego evalúa las condiciones de la Tabla 2.

Señal	Condición	Efecto
`rawD`	El slot 0 escribe un registro ($\neq$ x0) que el slot 1 lee. El valor recién se calcula en EX, en el mismo ciclo, y no se puede adelantar.	solo slot 0
`memD`	Ambas instrucciones son `lw`/`sw`: hay un solo puerto de memoria.	solo slot 0
`ctrlD`	Ambas instrucciones son `beq`/`jal`: hay una sola Branch Unit.	solo slot 0
`luseD_1`	El slot 1 lee el destino de un `lw` que está en EX (en cualquier vía).	solo slot 0
`LoadUseD_0`	El slot 0 lee el destino de un `lw` que está en EX (en cualquier vía).	stall (ninguna)

Cuadro 2: Condiciones evaluadas por la Issue Unit.

Las cuatro primeras se combinan en una sola salida:

$$\texttt{IssueOneD} = \texttt{rawD} \vee \texttt{memD} \vee \texttt{ctrlD} \vee \texttt{luseD_1} \tag{4}$$

Hay entonces tres resultados posibles: si `LoadUseD_0`= 1 no se emite nada (stall); si `IssueOneD`= 1 se emite solo el slot 0; en otro caso se emiten las dos instrucciones. Nunca se emite el slot 1 sin el slot 0, de modo que el orden del programa se mantiene.

Un load-use en el slot 1 no necesita stall: basta con emitir solo el slot 0. La instrucción que espera se reagrupa, entra a EX un ciclo después y para entonces el `lw` ya está en WB, desde donde se adelanta. Tampoco se restringen las dependencias WAW ni WAR dentro del par: como las vías escriben en WB en el mismo ciclo y la vía 1 tiene prioridad, el resultado final es el correcto (Programa 5).

3.2. Asignación de recursos

Las dos ALU están siempre disponibles, por lo que dos operaciones aritméticas independientes se emiten juntas. Los recursos compartidos no se asignan a una vía fija, sino a la vía que los necesita en ese ciclo:

- **Puerto de memoria:** un `lw`/`sw` puede ir en el slot 0 o en el slot 1, junto con una instrucción ALU, y `MemLaneM` le da el puerto. El par ALU + `lw` se emite igual que `lw` + ALU (Programa 3, ciclos 7 y 8).
- **Branch Unit:** un `beq`/`jal` puede ir en cualquier slot, y `BrLaneE` selecciona sus operandos.

Así, la única razón estructural para separar un par es que ambas instrucciones necesiten el mismo recurso compartido.

3.3. Emisión parcial y reagrupamiento

Cuando `IssueOneD`= 1 (y no hay stall ni salto tomado), la Hazard Unit activa `RegroupD` y ocurren tres cosas en el mismo flanco:

1. `FlushE_1` convierte en burbuja la vía 1 de ID/EX: el slot 1 no entra a EX.
2. Los multiplexores del IF/ID cargan el slot 1 como nuevo slot 0 e `InstrF_0` como nuevo slot 1.
3. El PC avanza 4 en lugar de 8.

En el siguiente ciclo, la instrucción que esperaba forma par con la instrucción que la sigue en el programa, y si son independientes se emiten juntas. Si se usara un simple stall (repetir el mismo par), la instrucción que esperaba solo podría salir sola; el reagrupamiento evita ese ciclo con una sola instrucción y es la razón por la que los programas con muchas dependencias siguen emitiendo pares (Programa 6).

4. Forwarding, hazards y control de branches

4.1. Forwarding entre vías y etapas

Cada uno de los cuatro operandos en EX (`Rs1E_0`, `Rs2E_0`, `Rs1E_1`, `Rs2E_1`) se compara con los cuatro destinos posibles: `RdM_1`, `RdM_0`, `RdW_1` y `RdW_0`. La función `fwd` de `hazard.v` elige la fuente con este orden de prioridad:

```
if      (rs == 5'b0)            fwd = 3'b000;  // x0: Register File
else if ((rs == rdM1) & weM1)   fwd = 3'b100;  // MEM lane 1
else if ((rs == rdM0) & weM0)   fwd = 3'b011;  // MEM lane 0
else if ((rs == rdW1) & weW1)   fwd = 3'b010;  // WB lane 1
else if ((rs == rdW0) & weW0)   fwd = 3'b001;  // WB lane 0
else                            fwd = 3'b000;  // Register File
```

La prioridad sigue la antigüedad de los productores: MEM es más reciente que WB y, dentro de la misma etapa, la vía 1 es más joven que la vía 0. Así, cuando varias instrucciones escriben el mismo registro, siempre se adelanta el valor más reciente. En las waveforms las fuentes aparecen como RF, WB0, WB1, MEM0 y MEM1.

El forwarding desde MEM toma `ALUResultM`, por lo que el dato de un `lw` solo puede adelantarse desde WB. Por eso se necesita el stall por load-use.

4.2. Stalls por load-use

`LoadUseD_0` detecta que el slot 0 necesita el destino de un `lw` que está en EX, en la vía 0 o en la vía 1. En ese caso:

$$\texttt{lwStallD} = \texttt{LoadUseD_0} \wedge \neg\texttt{PCSrcE} \tag{5}$$

$$\texttt{StallF} = \texttt{StallD} = \texttt{lwStallD} \tag{6}$$

$$\texttt{FlushE_0},\ \texttt{FlushE_1} \leftarrow \texttt{lwStallD} \tag{7}$$

El PC y el par en Decode se congelan y se inserta una burbuja en las dos vías de EX. En el siguiente ciclo el `lw` está en MEM; cuando el par entra a EX, el `lw` ya está en WB y su

dato se adelanta con WB0 o WB1. Un salto tomado tiene prioridad sobre el stall, porque en ese caso el par en Decode es del camino incorrecto.

4.3. Control de branches y flushing

Los saltos se resuelven en EX. Cuando `PCSrcE`= 1, el PC toma `PCTargetE` y se descartan las instrucciones más jóvenes que el salto:

- **Decode y Fetch:** `FlushD` = `PCSrcE` limpia el par en IF/ID.
- **Execute:** `FlushE_0` y `FlushE_1` limpian ID/EX en ambas vías.
- **La otra instrucción del par del salto:** depende de su posición.
  - Salto en la vía 0 (`TakenE_0`= 1): la instrucción de la vía 1 es más joven y es del camino incorrecto. `FlushM_1` limpia `regM_1`, así que en el siguiente ciclo esa instrucción es una burbuja en MEM y no escribe nada.
  - Salto en la vía 1: la instrucción de la vía 0 es más antigua y debe ejecutarse, por lo que no se limpia.

Señal	Expresión
`StallF`, `StallD`	`LoadUseD_0` $\wedge$ $\neg$`PCSrcE`
`RegroupD`	`IssueOneD` $\wedge$ $\neg$`lwStallD` $\wedge$ $\neg$`PCSrcE`
`FlushD`	`PCSrcE`
`FlushE_0`	`lwStallD` $\vee$ `PCSrcE`
`FlushE_1`	`lwStallD` $\vee$ `PCSrcE` $\vee$ `IssueOneD`
`FlushM_1`	`TakenE_0`

Cuadro 3: Resumen de las señales de control de la Hazard Unit.

Un salto tomado cuesta dos ciclos, igual que en el pipeline, pero en Dual-Issue cada ciclo perdido equivale a dos instrucciones, así que la penalización relativa es mayor.

5. Programas de prueba y waveforms

Los programas están en `tb/programs/dualissue/`. Cada uno termina con `jal x0,0` (*halt*). Las waveforms se generaron con `latex/informe-e2/waves/`: el testbench `trace_tb.v` imprime las señales en cada ciclo y `waves.py` las dibuja. Cada figura muestra los ciclos hasta que el *halt* llega a WB. Las instrucciones aparecen desensambladas en cada etapa y vía (`InstrD_0`, `InstrE_1`, etc.); *bubble* indica una burbuja. Los ciclos 0 a 2 corresponden al reset.

5.1. Programa 1: dos operaciones ALU independientes

```
// E2 Program 1 - Two independent ALU operations issued together
// No pair has a dependency inside it, so every pair issues in one cycle (both ALUs busy).
// Expected: x5=10 x6=6 x7=7 x8=2 x9=12 x10=1 ; MEM[0]=10 [4]=6 [8]=12
00300093  //   0: addi x1, x0, 3          ; pair: ALU + ALU
00700113  //   4: addi x2, x0, 7
00A00193  //   8: addi x3, x0, 10         ; pair: ALU + ALU
00400213  //  12: addi x4, x0, 4
002082B3  //  16: add  x5, x1, x2         ; pair: 3+7 = 10
40418333  //  20: sub  x6, x3, x4         ; 10-4 = 6
0040E3B3  //  24: or   x7, x1, x4         ; pair: 3|4 = 7
00317433  //  28: and  x8, x2, x3         ; 7&10 = 2
0062C4B3  //  32: xor  x9, x5, x6         ; pair: 10^6 = 12
00322533  //  36: slt  x10, x4, x3        ; 4<10 = 1
00502023  //  40: sw   x5, 0(x0)          ; pair: MEM + ALU
00100593  //  44: addi x11, x0, 1
00602223  //  48: sw   x6, 4(x0)          ; pair: MEM + ALU
00200613  //  52: addi x12, x0, 2
00902423  //  56: sw   x9, 8(x0)
0000006F  //  60: jal  x0, halt
```

Programa 1 (`dualissue/alu_pair.mem`).

Análisis. Ningún par tiene dependencias internas, así que `IssueOneD` vale 0 todo el programa y cada par pasa completo a EX. Desde el ciclo 4 el PC avanza de 8 en 8 (`0x8`, `0x10`, `0x18`, ...). En los ciclos 4 a 8 las dos ALU trabajan a la vez: por ejemplo, en el ciclo 6 `add x5,x1,x2` calcula $3+7=10$ en la vía 0 y `sub x6,x3,x4` calcula $10-4=6$ en la vía 1. Aunque dentro de cada par no hay dependencias, sí las hay entre pares consecutivos, y se resuelven con forwarding desde ambas vías: en ese mismo ciclo `add x5` toma x1 de WB0 y x2 de WB1, y `sub x6` toma x3 de MEM0 y x4 de MEM1.

En los ciclos 9 y 10 se emiten pares `sw` + `addi`: el `sw` usa el puerto de memoria y el `addi` la ALU de la otra vía. Las escrituras en memoria (ciclos 10 a 12) son `MEM[0]`$=10$, `MEM[4]`$=6$ y `MEM[8]`$=12$, las esperadas. En WB, `RegWriteW_0` y `RegWriteW_1` se activan juntos desde el ciclo 6 y se escriben dos registros por ciclo (x1/x2, x3/x4, x5/x6, ...). El programa termina en 12 ciclos con 16 instrucciones (IPC = 1,33); el pipeline necesita 20.

5.2. Programa 2: RAW dentro del par y forwarding entre vías

```
// E2 Program 2 - RAW between the two candidates and forwarding between lanes/stages
// - (addi x1 / addi x2,x1): RAW inside the pair -> only slot 0 issues, addi x2 is regrouped
// - (add x4 / sub x5): add takes x2 from MEM lane 0 and x3 from MEM lane 1 (cross-lane);
//                       sub takes x1 from WB lane 0 and x3 from MEM lane 1
// - (or x6 / add x7): or takes x4 from MEM lane 0 and x2 from WB lane 0;
//                     add takes x5 from MEM lane 1 and x4 from MEM lane 0
// Expected: x1=5 x2=6 x3=2 x4=8 x5=3 x6=14 x7=11 ; MEM[0]=8 [4]=3 [8]=14 [12]=11
00500093  //   0: addi x1, x0, 5
00108113  //   4: addi x2, x1, 1          ; RAW with slot 0 -> waits one cycle (regroup)
00200193  //   8: addi x3, x0, 2          ; issues with addi x2
00310233  //  12: add  x4, x2, x3         ; x2: MEM lane 0, x3: MEM lane 1
403082B3  //  16: sub  x5, x1, x3         ; x1: WB lane 0,  x3: MEM lane 1
00226333  //  20: or   x6, x4, x2         ; x4: MEM lane 0, x2: WB lane 0
004283B3  //  24: add  x7, x5, x4         ; x5: MEM lane 1, x4: MEM lane 0
00402023  //  28: sw   x4, 0(x0)          ; sw + sw: one memory port -> one per cycle
00502223  //  32: sw   x5, 4(x0)
00602423  //  36: sw   x6, 8(x0)
```

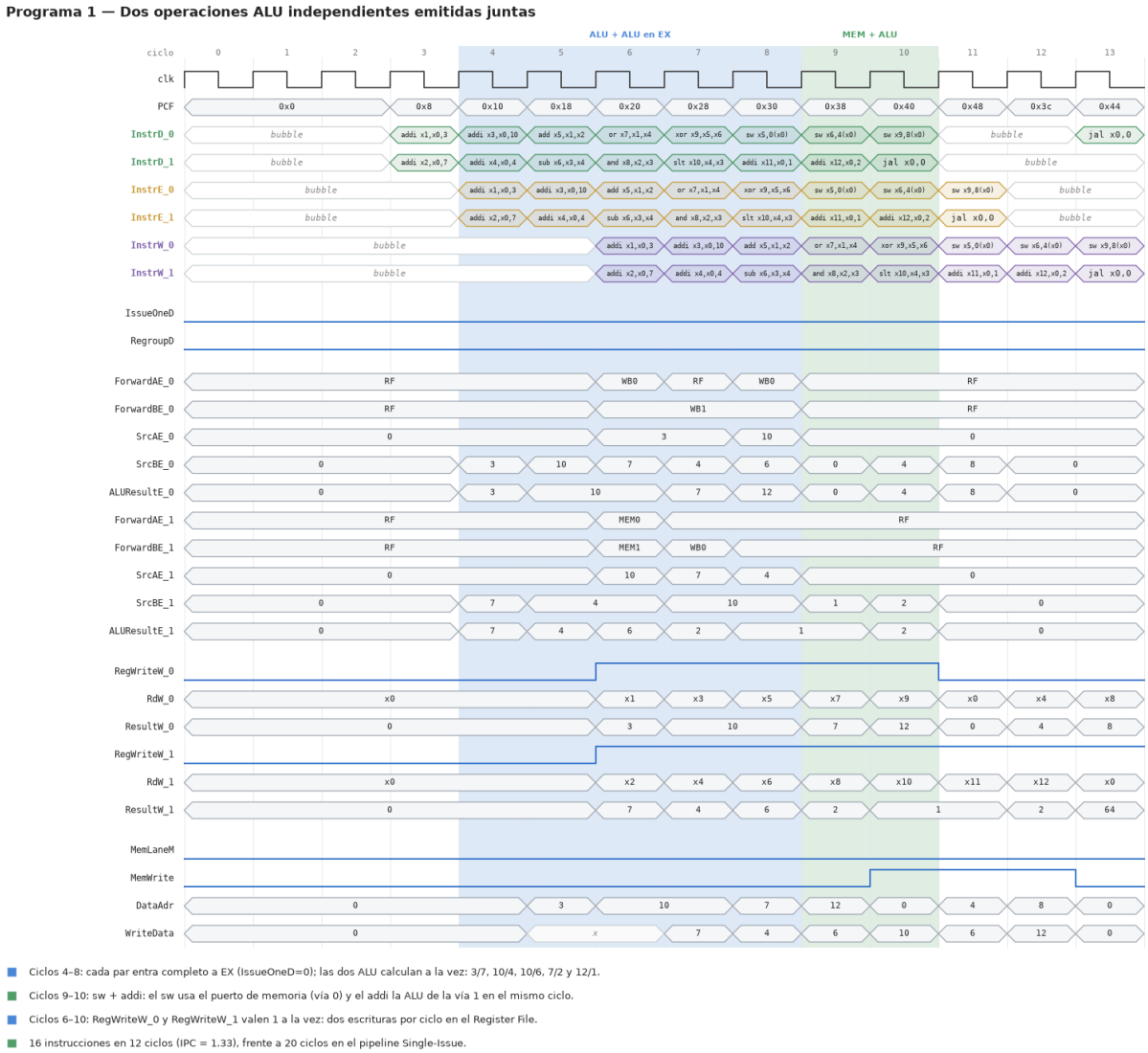


Figura 3: Waveform del Programa 1.

```
00702623  //  40: sw   x7, 12(x0)
0000006F  //  44: jal  x0, halt
```

Programa 2 (`dualissue/raw_forward.mem`).

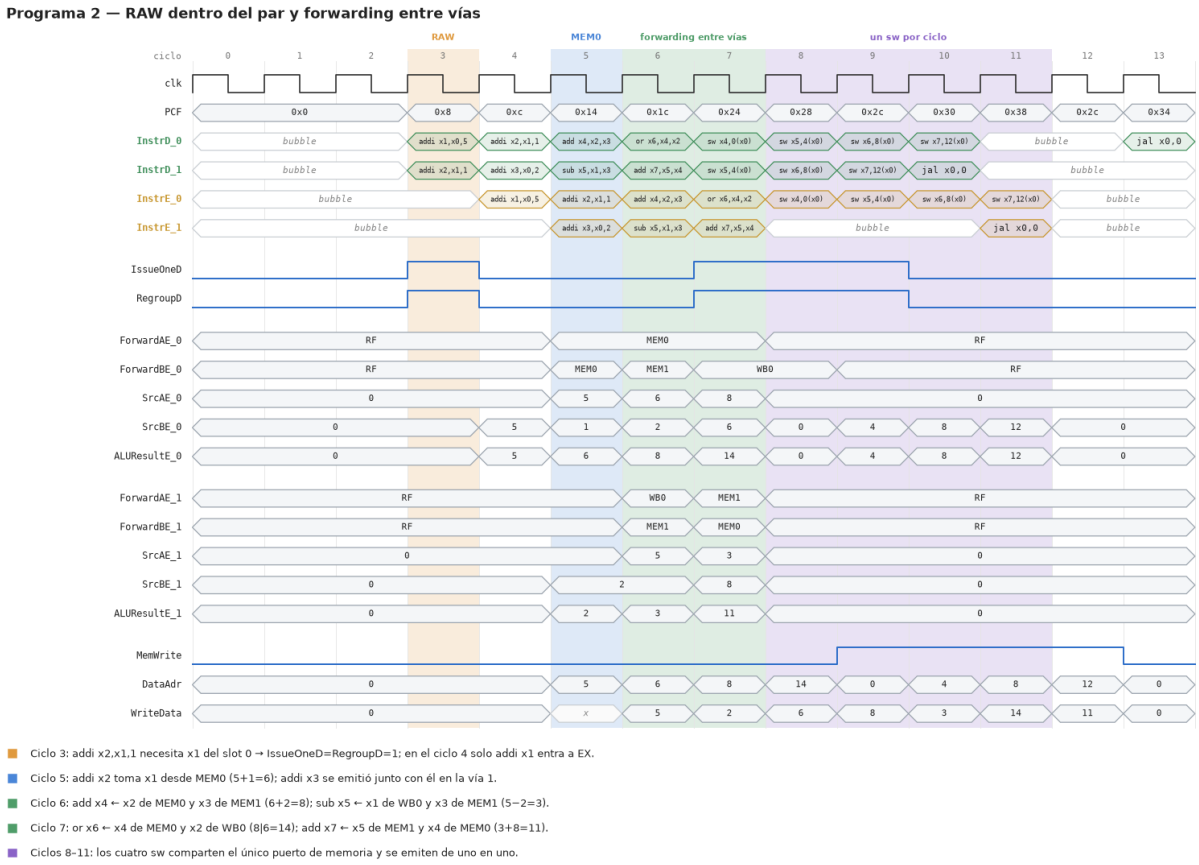


Figura 4: Waveform del Programa 2.

Análisis. En el ciclo 3 el par es (`addi x1,x0,5` | `addi x2,x1,1`). El slot 1 necesita x1, que el slot 0 recién calculará en EX, así que la Issue Unit activa `IssueOneD` y `RegroupD`. En el ciclo 4 solo `addi x1` entra a EX (`InstrE_1` es burbuja) y en Decode se forma el par (`addi x2` | `addi x3`). En el ciclo 5 `addi x2` toma x1 desde MEM0 y calcula $5 + 1 = 6$.

En los ciclos 6 y 7 se ve el forwarding cruzado entre vías. En el ciclo 6, `add x4,x2,x3` (vía 0) toma x2 de MEM0 y x3 de MEM1, que fue producido por la otra vía, y da 8. `sub x5,x1,x3` (vía 1) toma x1 de WB0 y x3 de MEM1, y da 3. En el ciclo 7, `or x6,x4,x2` usa MEM0 y WB0 ($8 \vee 6 = 14$), y `add x7,x5,x4` usa MEM1 para x5 y MEM0 para x4 ($3 + 8 = 11$). Las cuatro combinaciones entre vías y etapas aparecen en el programa.

Al final hay cuatro `sw` seguidos. Como hay un solo puerto de memoria, `IssueOneD` vale 1 en los ciclos 7 a 9 y cada `sw` entra solo a EX. Las escrituras de los ciclos 9 a 12 son `MEM[0]` = 8, `MEM[4]` = 3, `MEM[8]` = 14 y `MEM[12]` = 11.

5.3. Programa 3: load-use y conflictos del puerto de memoria

```
// E2 Program 3 - Load-use dependencies and conflicts on the single data memory port
// - sw + sw / lw + sw in the same pair -> structural conflict, issued in different cycles
// - lw + ALU and ALU + lw (independent) -> issued together, the memory port goes to either lane
// - lw followed by a consumer -> load-use stall (1 bubble in both lanes)
// - lw and its consumer in the same pair -> RAW: the consumer waits, then load-use stall
// Expected: x3=11 x4=22 x5=33 x6=12 x7=1 x8=34 ; MEM[0]=11 [4]=22 [8]=33 [12]=12 [16]=34
00B00093  //   0: addi x1, x0, 11
01600113  //   4: addi x2, x0, 22
00102023  //   8: sw   x1, 0(x0)          ; pair: sw + sw -> memory conflict
00202223  //  12: sw   x2, 4(x0)
00002183  //  16: lw   x3, 0(x0)          ; lw + independent ALU -> both issue (lane 0 uses memory)
00100393  //  20: addi x7, x0, 1
00900493  //  24: addi x9, x0, 9          ; ALU + lw -> both issue (lane 1 uses memory)
00402203  //  28: lw   x4, 4(x0)
004182B3  //  32: add  x5, x3, x4         ; load-use with x4 (lw in EX lane 1) -> stall
00108313  //  36: addi x6, x1, 1
00502423  //  40: sw   x5, 8(x0)
00802503  //  44: lw   x10, 8(x0)         ; lw + consumer in the same pair -> RAW, then load-use
00150413  //  48: addi x8, x10, 1
00602623  //  52: sw   x6, 12(x0)
00802823  //  56: sw   x8, 16(x0)
0000006F  //  60: jal  x0, halt
```

Programa 3 (`dualissue/memory.mem`).

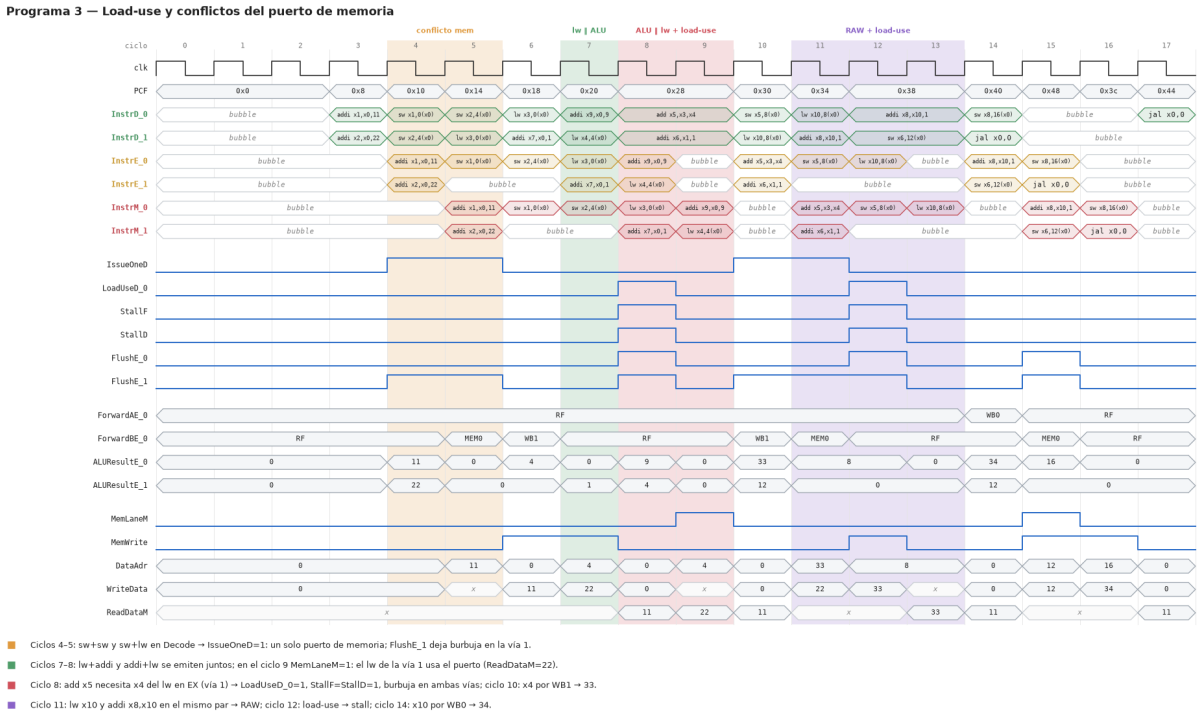


Figura 5: Waveform del Programa 3.

Análisis. En los ciclos 4 y 5 Decode tiene los pares (`sw` | `sw`) y (`sw` | `lw`). Ambas

instrucciones necesitan el puerto de memoria, por lo que `IssueOneD`= 1 y se emiten en ciclos distintos (`FlushE_1` deja la vía 1 vacía). En cambio, en el ciclo 7 entran juntos `lw x3` (vía 0) y `addi x7` (vía 1), y en el ciclo 8 `addi x9` (vía 0) y `lw x4` (vía 1). En el ciclo 9 `MemLaneM`= 1: el puerto se asigna a la vía 1, la dirección es 4 y `ReadDataM`= 22. Esto muestra que el orden ALU/memoria dentro del par no importa.

En el ciclo 8 `add x5,x3,x4` está en el slot 0 y necesita x4, que carga el `lw` de la vía 1 en EX. `LoadUseD_0` se activa, `StallF` y `StallD` congelan el PC (`0x28`) y el par, y `FlushE_0`/`FlushE_1` ponen burbujas en ambas vías (ciclo 9). En el ciclo 10 el `lw x4` ya está en WB de la vía 1, y `add x5` toma x4 por WB1 ($11 + 22 = 33$).

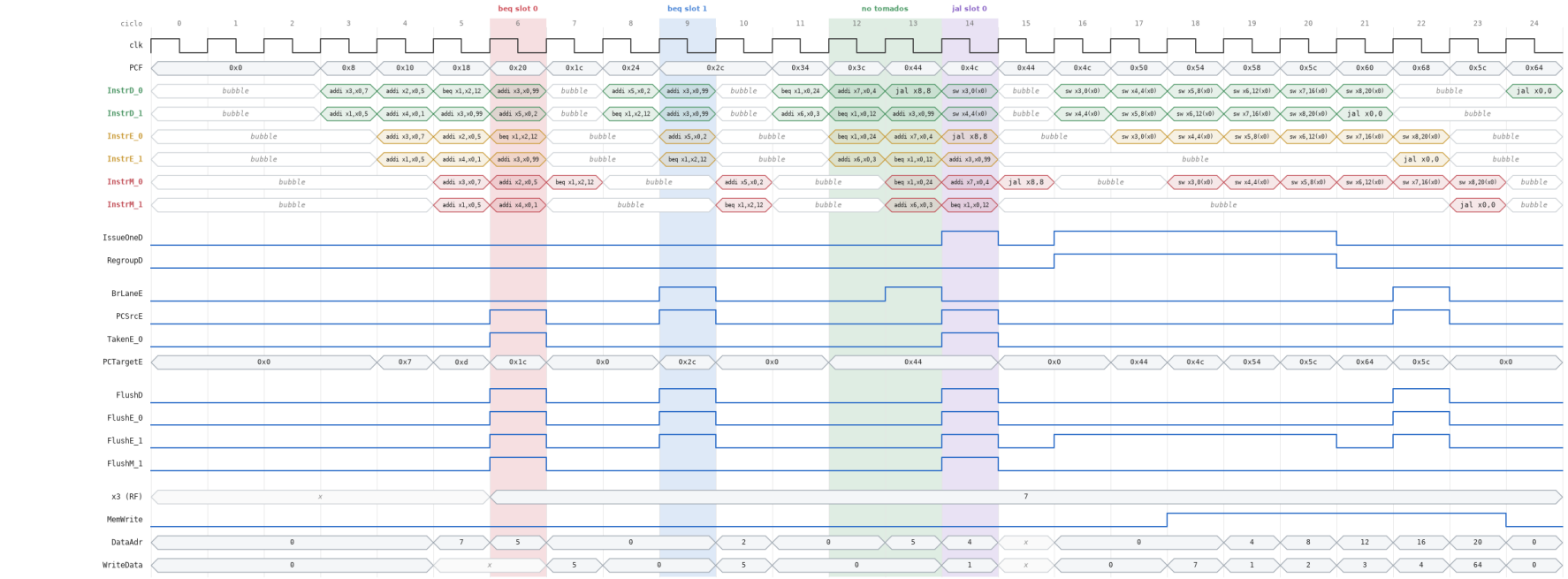
El último caso combina las dos situaciones. En el ciclo 11 el par (`lw x10` | `addi x8,x10,1`) tiene RAW, así que solo se emite el `lw`. En el ciclo 12 `addi x8` ya es el slot 0 y el `lw` está en EX, lo que produce un stall. En el ciclo 14 `addi x8` toma x10 de WB0 y calcula 34. El programa tiene 2 stalls, igual que el pipeline. Las escrituras son `MEM[0]`= 11, `MEM[4]`= 22, `MEM[8]`= 33, `MEM[12]`= 12 (`sw` en la vía 1, ciclo 15) y `MEM[16]`= 34.

5.4. Programa 4: branches en ambas posiciones y flushing

```
1  // E2 Program 4 - Branches in the first and second position of the pair, flushing
2  // Wrong-path instructions write 99 into x3; they must never retire.
3  // Expected: x3=7 x4=1 x5=2 x6=3 x7=4 x8=64 ; MEM[0]=7 [4]=1 [8]=2 [12]=3 [16]=4
       [20]=64
4  00700193  //   0: addi x3, x0, 7          ; pair
5  00500093  //   4: addi x1, x0, 5
6  00500113  //   8: addi x2, x0, 5          ; pair
7  00100213  //  12: addi x4, x0, 1
8  00208663  //  16: beq  x1, x2, t1        ; pair: branch in slot 0, TAKEN -> the slot
       1 instruction is annulled
9  06300193  //  20: addi x3, x0, 99        ; younger, same pair: annulled
10 06300193  //  24: addi x3, x0, 99
11 00200293  //  28: addi x5, x0, 2         ; pair: branch in slot 1, TAKEN -> slot 0 is
       older and is kept
12 00208663  //  32: beq  x1, x2, t2
13 06300193  //  36: addi x3, x0, 99
14 06300193  //  40: addi x3, x0, 99
15 00008C63  //  44: beq  x1, x0, t3        ; pair: branch in slot 0 NOT taken -> slot 1
       executes
16 00300313  //  48: addi x6, x0, 3
17 00400393  //  52: addi x7, x0, 4         ; pair: branch in slot 1 NOT taken
18 00008663  //  56: beq  x1, x0, t3
19 0080046F  //  60: jal  x8, t3            ; jal in slot 0 (x8 = 64) -> flushes slot 1
20 06300193  //  64: addi x3, x0, 99
21 00302023  //  68: sw   x3, 0(x0)
22 00402223  //  72: sw   x4, 4(x0)
23 00502423  //  76: sw   x5, 8(x0)
24 00602623  //  80: sw   x6, 12(x0)
25 00702823  //  84: sw   x7, 16(x0)
26 00802A23  //  88: sw   x8, 20(x0)
27 0000006F  //  92: jal  x0, halt
```

Programa 4 (`dualissue/branch.mem`).

Programa 4 — Branches en ambas posiciones del par y flushing



- Ciclo 6: beq en la vía 0 tomado (TakenE_0=1): FlushD y FlushE limpian Decode y FlushM_1 anula el addi x3,99 de su mismo par.
- Ciclo 9: beq en la vía 1 tomado (BrLaneE=1, TakenE_0=0): addi x5, más antiguo, sigue a MEM (InstrM_0 en el ciclo 10).
- Ciclos 12–13: beq no tomados en el slot 0 y en el slot 1: PCSrcE=0 y la otra instrucción del par continúa.
- Ciclo 14: jal x8 en la vía 0 anula el addi x3,99 de la vía 1. x3 se mantiene en 7 y MEM[0]=7.

Figura 6: Waveform del Programa 4.

Análisis. Las instrucciones del camino incorrecto escriben 99 en x3, así que si alguna llegara a WB, x3 dejaría de valer 7. La fila `x3 (RF)` se mantiene en 7 todo el programa.

Branch en el slot 0, tomado (ciclo 6). `beq x1,x2,12` está en la vía 0 de EX junto con `addi x3,x0,99` en la vía 1. Se activan `PCSrcE` y `TakenE_0`, y el PC salta a `0x1c`. `FlushD` y `FlushE` descartan las instrucciones de Decode, y `FlushM_1` anula el `addi` del mismo par: en el ciclo 7 `InstrM_1` es burbuja.

Branch en el slot 1, tomado (ciclo 9). El par es (`addi x5,x0,2` | `beq x1,x2,12`). Ahora `BrLaneE`= 1 y `TakenE_0`= 0, así que `FlushM_1` no se activa y `addi x5`, que es más antigua, continúa a MEM (ciclo 10). Solo se descartan las instrucciones más jóvenes, que están en Decode. El PC salta a `0x2c`.

Branches no tomados (ciclos 12 y 13). `beq x1,x0` no se cumple (x1= 5). En el ciclo 12 está en el slot 0 y en el 13 en el slot 1 (`BrLaneE`= 1). En ambos casos `PCSrcE`= 0, la otra instrucción del par (`addi x6`, `addi x7`) se ejecuta normalmente y el PC sigue avanzando de 8 en 8.

jal en el slot 0 (ciclo 14). `jal x8,8` es un salto incondicional en la vía 0: anula el `addi x3,99` de su par (`FlushM_1`) y guarda la dirección de retorno $x8 = 64$. Al final se escriben `MEM[0]`= 7, `[4]`= 1, `[8]`= 2, `[12]`= 3, `[16]`= 4 y `[20]`= 64, todos correctos.

5.5. Programa 5: dos escrituras en el Register File

```
1  // E2 Program 5 - Two writes into the Register File in the same cycle
2  // - pairs that write two different registers in the same WB cycle
3  // - a pair that writes the SAME register: the younger one (lane 1) must win,
4  //   in the register file and in the forwarding paths
5  // Expected: x1=1 x2=2 x3=30 x4=30 x5=31 ; MEM[0]=30 [4]=31 [8]=3
6  00100093  //   0: addi x1, x0, 1         ; pair: two writes (x1, x2) in the same
       cycle
7  00200113  //   4: addi x2, x0, 2
8  00A00193  //   8: addi x3, x0, 10        ; pair: WAW on x3 -> x3 = 30 (lane 1 is
       younger)
9  01E00193  //  12: addi x3, x0, 30
10 00018233  //  16: add  x4, x3, x0        ; x3 forwarded from MEM: lane 1 has priority
       -> 30
11 00000013  //  20: nop
12 00118293  //  24: addi x5, x3, 1         ; x3 forwarded from WB: lane 1 has priority
       -> 31
13 00208333  //  28: add  x6, x1, x2        ; 3
14 00402023  //  32: sw   x4, 0(x0)
15 00502223  //  36: sw   x5, 4(x0)
16 00602423  //  40: sw   x6, 8(x0)
17 0000006F  //  44: jal  x0, halt
```

Programa 5 (`dualissue/two_writes.mem`).

Análisis. En el ciclo 6 los dos `addi` del primer par llegan juntos a WB: `RegWriteW_0`=`RegWriteW_1`= 1, con x1= 1 por el puerto de la vía 0 y x2= 2 por el de la vía 1, en el mismo flanco de bajada.

El segundo par escribe el mismo registro (`addi x3,x0,10` | `addi x3,x0,30`). Como no hay RAW, se emite junto, y el valor correcto es el de la instrucción más joven, 30. Esto se cumple en tres lugares:

- **Forwarding desde MEM (ciclo 6):** `add x4,x3,x0` encuentra x3 en MEM0 y MEM1 a la vez; la prioridad elige MEM1 y `ALUResultE_0`= 30.

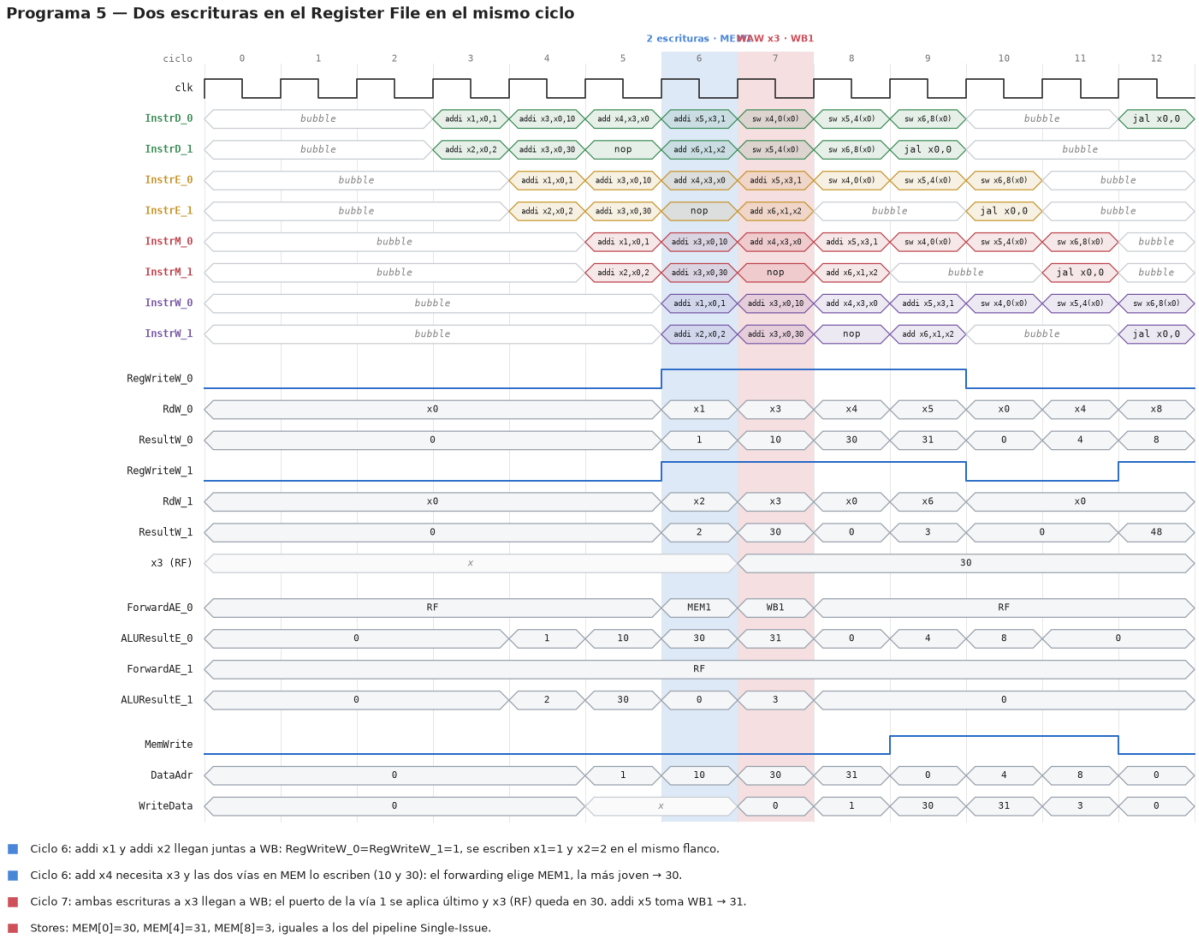


Figura 7: Waveform del Programa 5.

- **Escritura (ciclo 7):** ambos puertos escriben x3 (`ResultW_0`= 10, `ResultW_1`= 30); el de la vía 1 se aplica último y `x3 (RF)` queda en 30.
- **Forwarding desde WB (ciclo 7):** `addi x5,x3,1` encuentra x3 en WB0 y WB1; elige WB1 y calcula 31.

Las escrituras en memoria son `MEM[0]`= 30, `MEM[4]`= 31 y `MEM[8]`= 3.

5.6. Programa 6: reagrupamiento y orden del programa

```
1  // E2 Program 6 - Regrouping after a partial issue, program order, no lost/duplicated
       instructions
2  // A chain of dependent increments mixed with independent ones: many pairs issue only
       slot 0,
3  // and the waiting instruction is regrouped with the next one. Each sw stores the
       running value,
4  // so a lost or duplicated instruction would change the stored sequence.
5  // Expected: MEM[0]=1 [4]=2 [8]=3 [12]=4 [16]=5 [20]=6 ; x1=6 x2=100 x3=200 x4=300
6  00100093  //   0: addi x1, x0, 1
7  00102023  //   4: sw   x1, 0(x0)          ; RAW with slot 0 -> waits, regrouped
8  00108093  //   8: addi x1, x1, 1
9  06400113  //  12: addi x2, x0, 100        ; independent: issues with the instruction
       before
10 00102223  //  16: sw   x1, 4(x0)
11 00108093  //  20: addi x1, x1, 1
12 00102423  //  24: sw   x1, 8(x0)
13 00108093  //  28: addi x1, x1, 1
14 0C800193  //  32: addi x3, x0, 200
15 00102623  //  36: sw   x1, 12(x0)
16 00108093  //  40: addi x1, x1, 1
17 00102823  //  44: sw   x1, 16(x0)
18 12C00213  //  48: addi x4, x0, 300
19 00108093  //  52: addi x1, x1, 1
20 00102A23  //  56: sw   x1, 20(x0)
21 0000006F  //  60: jal  x0, halt
```

Programa 6 (`dualissue/regroup.mem`).

Análisis. El programa alterna una cadena de incrementos dependientes de x1 con instrucciones independientes, de modo que muchos pares no pueden emitirse completos. En el ciclo 3 el par es (`addi x1,x0,1` @`0x0` | `sw x1,0(x0)` @`0x4`) y tiene RAW. `RegroupD`= 1, el PC avanza solo 4 (`0x8`→`0xc`) y en el ciclo 4 el nuevo par es (`sw x1,0` @`0x4` | `addi x1,x1,1` @`0x8`): la instrucción que esperaba pasó al slot 0 y la primera que se buscó (`InstrF_0` del ciclo 3) pasó al slot 1. Ese par se emite completo en el ciclo 5.

El reagrupamiento se repite en los ciclos 6, 9 y 11. En todo momento `PCD_1`=`PCD_0`+4 y los valores de `PCD_0` solo crecen (`0x0`, `0x4`, `0xc`, `0x14`, `0x18`, ...), sin saltos ni repeticiones, lo que confirma que no se pierden ni se duplican instrucciones. También se observa un `sw` en el slot 1: en el ciclo 6 `sw x1,4(x0)` entra a EX por la vía 1, toma x1 de MEM1 y en el ciclo 7 usa el puerto de memoria (`MemLaneM`= 1).

Cada `sw` guarda el valor actual de x1, así que una instrucción perdida o repetida cambiaría la secuencia guardada. `WriteData` toma los valores 1, 2, 3, 4, 5 y 6, en ese orden y en las direcciones 0 a 20.

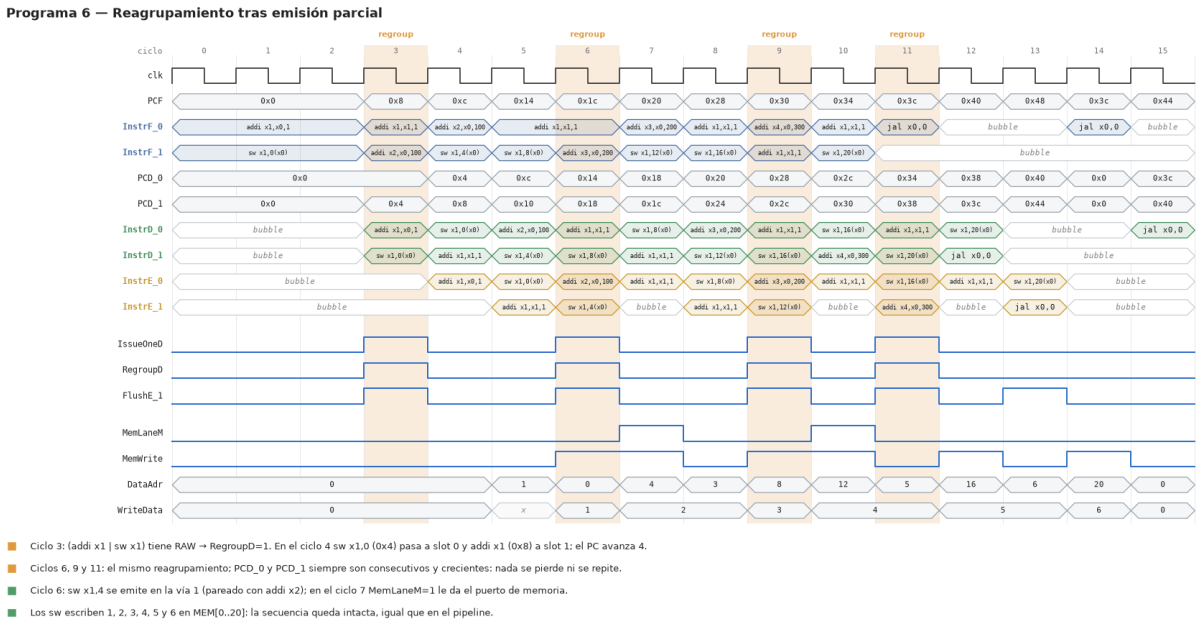


Figura 8: Waveform del Programa 6.

5.7. Programa 7: conflicto en la Branch Unit y jal en el slot 1

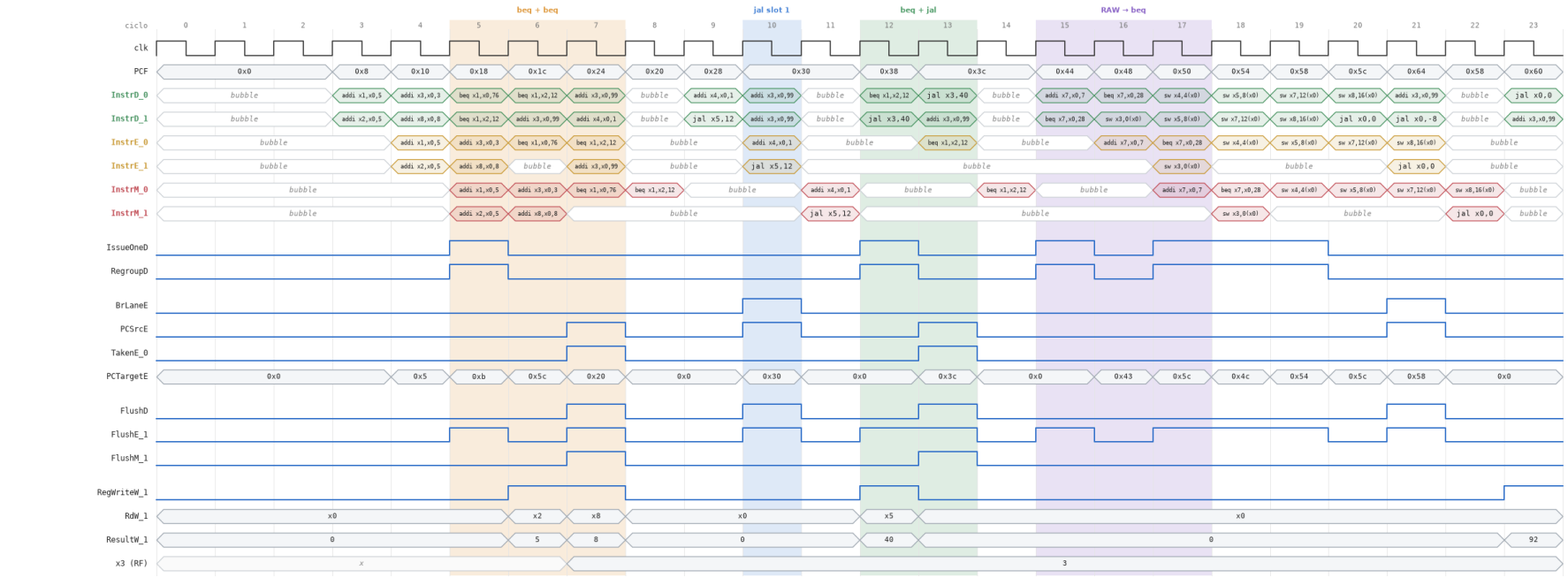
Este programa completa los casos de control: dos saltos en el mismo par, un `jal` tomado en el slot 1 y un `beq` que depende de la otra instrucción de su par.

```
1  // E2 Program 7 - Single branch unit: two control instructions in the same pair, jal
       in slot 1
2  // - beq + beq and beq + jal in the same pair -> structural conflict on the branch
       unit,
3  //   issued in different cycles (the younger one is regrouped as the next slot 0)
4  // - jal in slot 1 taken -> slot 0 (older) is kept, younger instructions are flushed
5  // - branch in slot 1 that depends on slot 0 (RAW) -> issued alone in the next cycle
6  // Wrong-path instructions write x3; they must never retire.
7  // Expected: x3=3 x4=1 x5=40 x7=7 x8=8 ; MEM[0]=3 [4]=1 [8]=40 [12]=7 [16]=8
8  00500093  //    0:        addi x1, x0, 5          ; pair
9  00500113  //    4:        addi x2, x0, 5
10 00300193  //    8:        addi x3, x0, 3          ; pair
11 00800413  //   12:        addi x8, x0, 8
12 04008663  //   16:        beq  x1, x0, bad        ; pair: TWO branches -> single branch
       unit: only slot 0 issues; NOT taken
13 00208663  //   20:        beq  x1, x2, t1         ; waits, regrouped as slot 0 in the
       next cycle; TAKEN
14 06300193  //   24:        addi x3, x0, 99         ; wrong path
15 06300193  //   28:        addi x3, x0, 99         ; wrong path
16 00100213  //   32: t1:    addi x4, x0, 1          ; pair: jal in slot 1 TAKEN -> slot 0
       (older) is kept
17 00C002EF  //   36:        jal  x5, t2             ; x5 = return address
18 06300193  //   40:        addi x3, x0, 99         ; wrong path
19 06300193  //   44:        addi x3, x0, 99         ; wrong path
20 00208663  //   48: t2:    beq  x1, x2, t3         ; pair: beq + jal -> only the beq
       issues; beq TAKEN
21 028001EF  //   52:        jal  x3, bad            ; wrong path: waits in Decode and is
       flushed (would write x3)
22 06300193  //   56:        addi x3, x0, 99         ; wrong path
23 00700393  //   60: t3:    addi x7, x0, 7          ; pair: branch in slot 1 needs x7 (
       RAW) -> waits, regrouped
24 00038E63  //   64:        beq  x7, x0, bad        ; NOT taken (x7 forwarded)
25 00302023  //   68:        sw   x3, 0(x0)
```

```
00402223  //  72:        sw   x4, 4(x0)
00502423  //  76:        sw   x5, 8(x0)
00702623  //  80:        sw   x7, 12(x0)
00802823  //  84:        sw   x8, 16(x0)
0000006F  //  88: halt:  jal  x0, halt
06300193  //  92: bad:   addi x3, x0, 99          ; only reached on a wrong branch
FF9FF06F  //  96:        jal  x0, halt
```

Programa 7 (`dualissue/branch_conflict.mem`).

Programa 7 — Una sola Branch Unit: dos saltos en el par y jal en la segunda posición



- Ciclo 5: dos beq en el par → una sola Branch Unit: IssueOneD=1. El segundo beq se reagrupa y se resuelve tomado en el ciclo 7.
- Ciclo 10: jal x5 en la vía 1 tomado (BrLaneE=1): addi x4 (vía 0) se conserva; x5=40 se escribe por WB1 en el ciclo 12.
- Ciclos 12–13: beq + jal x3: solo entra el beq; al tomarse, FlushD descarta el jal x3 que esperaba en Decode (x3 sigue en 3).
- Ciclos 15–17: beq x7 depende de addi x7 en el mismo par → espera un ciclo y toma x7 por forwarding (no tomado).

Figura 9: Waveform del Programa 7.

Análisis. En el ciclo 5 el par es (`beq x1,x0` | `beq x1,x2`). Como hay una sola Branch Unit, `IssueOneD`= 1 y solo pasa el primero (no tomado, ciclo 6). El segundo se reagrupa como slot 0, entra a EX en el ciclo 7 y se toma: el PC va a `0x20`, se limpia Decode y `FlushM_1` anula el `addi x3,99` que lo acompañaba en la vía 1.

En el ciclo 10 el par en EX es (`addi x4` | `jal x5,12`). El salto está en la vía 1 (`BrLaneE`= 1, `TakenE_0`= 0), así que `addi x4` se conserva y llega a MEM en el ciclo 11. El `jal` escribe su dirección de retorno por la vía 1: `RdW_1`=x5, `ResultW_1`= 40 en el ciclo 12.

En el ciclo 12 el par (`beq x1,x2` | `jal x3,40`) vuelve a tener dos saltos y solo entra el `beq`. En el ciclo 13 el `beq` se toma y `FlushD` descarta el `jal x3` que esperaba en Decode. Ese `jal` habría escrito x3, pero x3 se mantiene en 3. Por último, en el ciclo 15 `beq x7,x0` depende de `addi x7` en el mismo par: espera un ciclo, entra a EX en el ciclo 17 con x7 adelantado desde MEM0 y no se toma. Las escrituras son `MEM[0]`= 3, `[4]`= 1, `[8]`= 40, `[12]`= 7 y `[16]`= 8.

5.8. Resumen de señales

Señal	**Qué indica**
`InstrF/D/E/M/W_0/1`	Instrucción en cada etapa y vía (solo para depurar).
`PCD_0`, `PCD_1`	Dirección de cada slot del par en Decode.
`IssueOneD`	La Issue Unit solo deja pasar el slot 0.
`LoadUseD_0`	El slot 0 depende de un `lw` en EX (stall).
`RegroupD`	El slot 1 pasa a slot 0; el PC avanza 4.
`StallF`, `StallD`	Congelan el PC y el IF/ID.
`FlushD`, `FlushE_0/1`, `FlushM_1`	Anulan Decode, EX de cada vía y MEM de la vía 1.
`ForwardAE/BE_0/1`	Fuente de cada operando: RF, WB0, WB1, MEM0 o MEM1.
`SrcAE/SrcBE/ALUResultE_0/1`	Operandos y resultado de cada ALU.
`BrLaneE`, `PCSrcE`, `TakenE_0`, `PCTargetE`	Vía del salto en EX, si se toma, si está en la vía 0 y a dónde va.
`MemLaneM`, `MemWrite`, `DataAdr`, `WriteData`, `ReadDataM`	Vía dueña del puerto de memoria y el acceso.
`RegWriteW/RdW/ResultW_0/1`	Las dos escrituras en el Register File.

Cuadro 4: Señales observadas en las waveforms.

6. Comparación preliminar frente al Single-Issue

6.1. Metodología

Ambos diseños se simulan con el mismo testbench (`tb/perf_tb.v`; el Dual-Issue se compila con `-DDUAL`) y los mismos programas, sin modificarlos. El criterio de medición es el mismo en los dos: se cuenta desde el primer ciclo después del reset hasta el ciclo en que el *halt* (`jal x0,0`) llega a WB. Las instrucciones retiradas son las que llegan a WB (sin contar burbujas), incluido el *halt*. El script `tb/run_all.sh` ejecuta todos los programas en las dos arquitecturas, compara el estado final (los 31 registros y la memoria de datos) y calcula las métricas:

$$\text{IPC} = \frac{\text{instrucciones}}{\text{ciclos}}, \qquad \text{CPI} = \frac{\text{ciclos}}{\text{instrucciones}}, \qquad \text{Speedup}_{A\to B} = \frac{\text{ciclos}(A)}{\text{ciclos}(B)}$$

Además de los programas de esta entrega se incluyen los cuatro de E1 y un pequeño benchmark, `loop.mem`, que llena un arreglo de 8 palabras con un bucle y luego lo suma con otro (`lw`, `add`, `beq`, `jal`).

6.2. Resultados

		A: Pipeline		**B: Dual-Issue**			**Eventos en B (ciclos)**				
Programa	**Instr.**	**Ciclos**	**IPC**	**Ciclos**	**IPC**	**CPI**	**Dual**	**Regr.**	**Stall**	**Flush**	**Speedup**
P1 `alu_pair`	16	20	0.800	12	1.333	0.750	8	0	0	1	1.667
P2 `raw_forward`	12	16	0.750	12	1.000	1.000	4	4	0	1	1.333
P3 `memory`	16	22	0.727	16	1.000	1.000	6	4	2	1	1.375
P4 `branch`	19	29	0.655	23	0.826	1.211	8	5	0	4	1.261
P5 `two_writes`	12	16	0.750	11	1.091	0.917	5	2	0	1	1.455
P6 `regroup`	16	20	0.800	14	1.143	0.875	6	4	0	1	1.429
P7 `branch_conflict`	17	27	0.630	22	0.773	1.294	6	6	0	4	1.227
E1 `isa`	14	18	0.778	13	1.077	0.929	5	4	0	1	1.385
E1 `forwarding`	8	12	0.667	11	0.727	1.375	1	6	0	1	1.091
E1 `stall`	9	15	0.600	14	0.643	1.556	1	7	2	1	1.071
E1 `flush`	8	16	0.500	14	0.571	1.750	4	2	0	3	1.143
`loop` (benchmark)	89	129	0.690	93	0.957	1.045	45	8	0	19	1.387

Cuadro 5: Ciclos, IPC y speedup del Dual-Issue (B) frente al pipeline Single-Issue (A). *Dual*: ciclos con dos instrucciones en EX; *Regr.*: ciclos con reagrupamiento; *Flush*: ciclos con `FlushD` (incluye el *halt*).

En los 12 programas, `run_all.sh` reporta **OK**: el pipeline y el Dual-Issue terminan con los mismos registros y la misma memoria de datos.

6.3. Análisis

El Dual-Issue es más rápido en todos los programas, con speedups entre 1.07 y 1.67. La mejora depende de cuánto paralelismo tenga cada programa:

- **Instrucciones independientes.** En P1 casi todos los pares se emiten juntos (8 ciclos con dos instrucciones en EX) y se obtiene el mayor speedup, 1.67. Aun así, el IPC es 1.33 y no 2, porque los programas son cortos y los 4 ciclos de llenado del pipeline y el *halt* pesan mucho: 16 instrucciones necesitan al menos $8+4 = 12$ ciclos.
- **Dependencias.** Los programas de E1 `forwarding` y `stall` son cadenas en las que cada instrucción depende de la anterior. Casi ningún par puede emitirse completo (6 y 7 reagrupamientos), así que el Dual-Issue se comporta como el pipeline (speedup de 1.09 y 1.07). El paralelismo disponible lo limita el programa, no el hardware.
- **Recursos compartidos.** Los `sw` consecutivos de P2, P3 y P4 se emiten de a uno por el único puerto de memoria. P7 pierde ciclos adicionales porque dos saltos no pueden ir en el mismo par.
- **Saltos.** Es la limitación más importante en código con bucles. En `loop` hay 19 saltos tomados (18 del bucle más el *halt*), cada uno resuelto en EX con 2 ciclos perdidos: unos 38 de los 93 ciclos se pierden por flush. Por eso el IPC queda en 0.96 aunque en 45 ciclos se ejecutan dos instrucciones a la vez. En el pipeline un salto tomado desperdicia 2 instrucciones; en el Dual-Issue desperdicia hasta 4.

En resumen, el procesador aprovecha el paralelismo cuando el código lo permite. Lo que impide acercarse al IPC ideal de 2 son, en este orden, los saltos tomados, las dependencias RAW entre instrucciones vecinas y los conflictos en los recursos compartidos. Estas mediciones servirán como referencia (arquitectura B) para evaluar la optimización de la entrega final.

7. Conclusiones

- Se completó el procesador Dual-Issue In-Order: dos vías con ALU propia hasta WB, Register File de 4 lecturas y 2 escrituras, un único puerto de memoria y una única Branch Unit, ambos asignados dinámicamente a la vía que los necesita.
- La Issue Unit emite dos instrucciones, una o ninguna según RAW dentro del par, conflictos de memoria o de saltos y load-use. Cuando emite solo una, el reagrupamiento combina la instrucción pendiente con la siguiente sin perder ni repetir instrucciones.
- La Hazard Unit adelanta datos desde las dos vías en MEM y WB, con prioridad para el productor más joven, y anula únicamente las instrucciones más jóvenes que un salto tomado, conservando la instrucción más antigua cuando el salto está en el slot 1.
- Los siete programas de prueba y los cuatro de E1 producen el mismo estado final en el pipeline y en el Dual-Issue. El speedup va de 1.07 a 1.67, y los saltos tomados son la principal causa de pérdida de rendimiento.